

CS 2308: Foundations of Computer Science II

Instructor: Dr. Tanzima Islam
Comal 307A
tanzima@txstate.edu

Virtual Office Hours: Check Canvas

Learning Management System: Class materials will be available on **Canvas**.

Course Format (Mode of Delivery):

This is a **Face to Face** course, class sessions take place as follows:

11:00 am - 12:20 pm MW

Required Programming Environment: zyBooks

Option 1:

1. Sign in or create an account at learn.zybooks.com
2. Enter zyBook code: TXSTATECS2308IslamFall2025
3. Click on Subscribe. (or get an access code from the TXST Bookstore)

Option 2:

1. Click any zyBooks assignment link on CANVAS
(Do not go to the zyBooks website and create a new account)
2. Subscribe

Only use one of the options to get started!

- Issues: contact support@zybooks.com.
- You will use this online environment to submit programming assignments.
- It will have built-in test cases used to grade your programs (results upon submission).
- It also contains many practice problems.

Classroom Behavior:

- We want to establish a collaborative and engaging classroom environment. Your participation is important.
- We will be respectful toward everyone and responsive to the instructor's questions.
- We will raise our hands to ask questions.
- We will complete our labs in class and check with the instructor, ask for help.

Course Description:

Fundamentals of object-oriented programming. Introduction to abstract data types (ADTs) including lists, stacks, and queues. Searching and sorting. Pointers and dynamic memory allocation. A continuation of CS 1428.

Prerequisites:

- * C or higher in CS 1428: Foundations of Computer Science I
- * Know the basics of C or C++ programming syntax.

Course Objectives:

At the end of the course, the students should be able to:

1. Describe and demonstrate at least two different algorithms for searching and at least two different algorithms for sorting.
2. Implement a divide-and-conquer algorithm to solve an appropriate problem (binary search).
3. State the time/space efficiency of various algorithms (using one of 6 categories of mathematical functions).
4. List the 6 categories of mathematical functions used in analyzing algorithms in order from slowest to fastest growing.
5. Read and write C++ code that uses pointer variables and memory operations (new, &, *, delete), including pointers to arrays, structures, and objects and the -> operator.
6. Write C++ code that resizes an array using dynamic memory allocation.
7. Write C++ code that deletes dynamically allocated memory to avoid memory leaks.
8. Describe the basic concepts of object-oriented programming.
9. Design, implement, test, and debug simple programs (using objects) in an object-oriented programming language (C++).
10. Describe how the class mechanism supports encapsulation and information hiding.
11. Develop (implement) programs using multiple classes and arrays of objects
12. Develop and use appropriate algorithms, especially for processing lists (insert, remove, search, sort, etc.)
13. Describe structured programming in terms of modules and functions.
14. Develop (implement) programs with source code separated into multiple files, including header (.h) files
15. Create, compile, and run a C++ program in a unix style command-line environment
16. Develop (Implement) C++ programs that create and use simple linked-lists, including code to insert into, delete from, and traverse a linked list structure.
17. Compare and contrast the costs and benefits of dynamic and static data structure implementations.
18. Describe the principle of the Abstract Data Type (ADT) and, in particular, explain the benefits of separation of interface and implementation.
19. Implement user-defined data structures in a high-level language.
20. Implement the list, stack, and queue ADT using arrays and linked lists.
21. Write programs that use each of these data structures: linked lists, stacks, and queues.

Grading:	Participation:	10%	In-class lab, answering questions, asking questions, watching pre-class video lectures
	Quizzes	15%	lowest one is dropped
	Programming Assignments:	35%	lowest one is dropped
	Midterm:	15%	Check Canvas
	Final Exam (comprehensive):	25%	Check Canvas

	Topic	Dates
Unit 1	Functions, Arrays & Structs	Week 1 Week 2
Unit 2	Searching, Sorting & Analysis	Week 3 Week 4
Unit 3	Pointers & Dynamic Memory Allocation	Week 5 Week 6
Unit 4	Intro to Classes	Week 7 Week 8
Unit 5	Linked Lists	Week 9 Week 10
Unit 6	Stacks & Queues	Week 11 Week 12

Class Preparation Requirement:

Before class, you are expected to complete assigned **readings or pre-recorded video lectures**. These prepare you for in-class discussions and quizzes.

In class, I will:

- Review the main concepts
- Answer questions
- Work through examples

Class Participation:

Please bring a web-enabled laptop or tablet to work on in-class labs. If you do not have such a device, you may pair up with another student or use one of the CS labs. **In-class labs are always scheduled at the end of class, and there is no lecture afterward.**

Labs are graded on a **complete/incomplete basis**. To receive credit, you must finish and submit the lab. These labs are your chance to practice implementing the concepts we cover in class, and your assignments will build directly on these components.

Participation also includes **answering or asking questions** during class discussions.

Coding Quizzes:

Starting **after Wednesday, September 3**, there will be regular quizzes on Canvas. Quiz dates are flexible but will always be announced in advance.

The quizzes may include multiple-choice, true/false, or short programming tasks such as writing code, modifying existing code, or identifying errors in C++ programs. The exact format may vary from week to week, depending on class needs.

Communication: Canvas, Slack, and Email

I will post **official course announcements** (deadlines, lecture slides, quizzes, assignments, reading materials, and reminders) on **Canvas Announcements**. Important updates about exams, deadlines, or schedule changes will also be shared there.

For **questions and discussion**, we will use **Slack**. All course-related communication (questions about class content, assignments, exams, etc.) should be posted in the Slack channel: **#Fall2025-cs2308**. You are all by default added to this channel.

Using this shared channel ensures that both the TA and I can respond quickly, and that everyone benefits from seeing the discussion.

- **Always post in the #Fall2025-cs2308 channel** for course-related questions.
- **Direct messages** should only be used if your question includes **personal or sensitive information** (e.g., grades, accommodations).

Email will be used **rarely** and only for official or administrative purposes.

Email Response Time

I do my best to respond to emails promptly. You can expect a reply within **24–48 hours on weekdays**. If you do not receive a response within that time, feel free to re-send your message.

Please note:

- I do not guarantee responses over weekends.
- Emails sent less than **24 hours before a project deadline** may not receive a response. This policy is in place to encourage you to start projects early.

Email Format

When emailing me, **please make sure to CC the TA** and use this subject line format:
[CS2308] A few descriptive words

Using this format ensures your message is not filtered out as spam. If your email concerns a group project, please **CC all group members**.

Attendance policy:

Attendance is encouraged but not mandatory. However, class activities and labs are designed for hands-on implementation with my guidance, and regular attendance will make it much easier to succeed. Participation points are tied to engagement, and while you can earn credit through activities such as asking/answering questions on Slack or watching the assigned pre-class video lectures, you cannot receive full participation credit without attending class in person.

Programming Assignments:

There will be one assignment for each unit.

- **Assignment environment:**

- The first assignment may be completed and submitted in the zyBooks platform.
- All later assignments must be developed in a **Linux environment** and submitted through zyBooks.

- **Grading:**

Assignments are graded on correctness, **meeting requirements as specified in the instructions**, and programming style.

- **Collaboration:**

You may work in groups of **up to 4 students** on all programming assignments. Each student must submit their own copy of the program. Programs submitted as group work must list **all contributors in the file header** and describe each member's contribution. Failure to do so will result in a **–30 point penalty**.

- **Restrictions:**

Do not include code obtained from the internet or any source other than what I provide during the current semester. Do not post your solutions online. Violations will result in a **0 for the assignment**.

Examinations/Tests:

- Tests will be either in-class on paper or on Canvas. The proper format will be communicated in advance.
- No makeup tests are available unless proper medical documentation is submitted.

Makeup Policy:

Missed quizzes and assignments cannot normally be made up. **Exceptions will be made only in two cases:**

1. **Medical emergencies** with proper documentation (doctor's note required).
2. **Other valid conflicts** that have been communicated to me **in advance** and approved beforehand.

If no communication is made prior to the deadline (except in documented medical emergencies), **no accommodation will be provided**.

HELP:

Other than the instructor's office hours, there are other places to obtain assistance:

- **CS Lab tutors** (good for help on Programming Assignments):
<https://cs.txstate.edu/resources/labs/tutoring/>
- **CLC tutoring** (good for help on understanding the material):
<https://hlsamp.cose.txstate.edu/clc/clc-tutoring.html>
- **TA's office hours** see Canvas for times and how to access.

Use of AI Tools and Internet Resources

You may use AI tools such as ChatGPT, Perplexity, Gemini, or Claude, as well as other internet resources, to **clarify your understanding**, ask for **simple explanations**, explore **basic examples**, or watch **animations/visualizations** that help you learn concepts such as arrays, searching, and sorting. However, you may **not** use AI to generate code and copy-paste it into your assignments. Copy-pasting code from AI or the internet is considered **academic dishonesty**. If AI provides a code example, you must **type it yourself** and understand every part of it. Typing ensures your brain engages with the logic and prevents your work from being flagged as AI-generated (AI tools often alter variable names or code structures in ways that are easy to detect).

If your work is suspected of being AI-generated, you may be asked to **recreate the entire program in person**. If you cannot do so, it will be treated as cheating and will result in a **0 on the assignment** and a report under the **Academic Honesty policy**.

When you use AI or online resources for explanations (not code), you must **cite the tool in a comment** (e.g., // Concept explanation adapted from ChatGPT). This protects you from suspicion and makes your learning process transparent.

Remember: **AI is a tutor, not a substitute for your own problem-solving skills**. If you do not develop the logic yourself, the AI becomes a crutch but not your brain—you must know what you want to implement and why. Use AI to support your learning, may be brainstorm ideas, but always do the work yourself.

Academic Honesty:

You are expected to follow the University's Academic Honor Code:

<https://www.txstate.edu/honorcodecouncil/Academic-Integrity.html>

- Submitting someone else's work as your own is dishonesty.
- Cheating on tests will result in a **0 on that test**.
- Using unapproved code sources or failing to cite collaborators will result in penalties as described above.
- **Copy-pasting code from any source — including AI tools (ChatGPT, Claude, Gemini, Perplexity, etc.), Stack Overflow, Google searches, or any other**

internet or external resource — is considered academic dishonesty. Please see the “Use of AI Tools and Internet Resources” section for detailed guidelines.

Withdrawals/drops:

You must follow the withdrawal and drop policy set up by the University:

<http://www.registrar.txstate.edu/registration/dropping-or-withdrawing.html>

Accommodations for students with disability:

Any student with needs requiring special accommodations should contact the office of disability services at the LBJ student center (5-5.1). Students who qualify for extra time for exams must take their test with ATSD and must schedule their test at the same time the test is given in class. Note: you must submit your request with ATSD at least 2 business days before the exam date!

Statements from the University on:

Classroom Civility, measures for health, wellness, and safety, the Bobcat Pledge, and compliance for health standards: See course home page on canvas for these.